

Propuesta: Sistema de Gestión Integral para StartUps

1. Análisis del caso de uso

Objetivo: Diseñar un sistema de uso interno, destinado a StartUps cuya unidad de negocio se basa en el diseño y fabricación de proyectos. Este sistema debe permitir:

- Llevar registro de los stakeholders (clientes, empleados, proveedores).
- Llevar registro de compras y ventas
- Llevar registro de proyectos, incluyendo:
 - Documentación, con control de versiones
 - Archivos, con control de versiones
 - Componentes
 - Implicados
- Llevar control de stock

Adicionalmente, este sistema podrá incorporar un agente de IA que permita al usuario hacer consultas y obtener respuestas en base a la información disponible del sistema. Por ejemplo

- Cuál es el stock disponible de ...?
- Que componentes hace falta conseguir para construir el proyecto ...?
- Haz un resumen del proyecto ...?

Este sistema soluciona el problema de no tener la información centralizada y organizada, lo que complica la trazabilidad de todos los aspectos del negocio, así como brindar la posibilidad de hacer consultas en lenguaje natural y obtener respuestas basadas en los datos propios.

En primera instancia, este sistema solo será utilizable por aquellos que pertenecen a la organización, presentando control de acceso en función de su participación en el proyecto y rol asignado.

Se deberá tener precaución a la hora de manejar el versionado de los documentos ya que se presenta el riesgo de perder información así como también brindar versiones obsoletas. Por otro lado, se debe tener cuidado con los componentes ya que son susceptibles a ser duplicados por accidente.

Para lograr adopción del sistema se deberá conveniar con los interesados para conocer el flujo de operaciones y adaptar el sistema a ellas y no viceversa. Además, se debe procurar que la interacción con el mismo sea simple.

2. Relevamiento de datos necesarios

Se identifica que se deberán almacenar los siguientes datos:

Datos Estructurados:

- Información de clientes
- Información de proveedores
- Información de empleados
- Metadatos de partes
- Metadatos de documentos
- Metadatos de archivos

Datos Semiestructurados:

- Descripción del proyecto
- Descripción de parte

Datos no estructurados:

- Planos
- Modelos 3D
- Archivos

Datos Operacionales

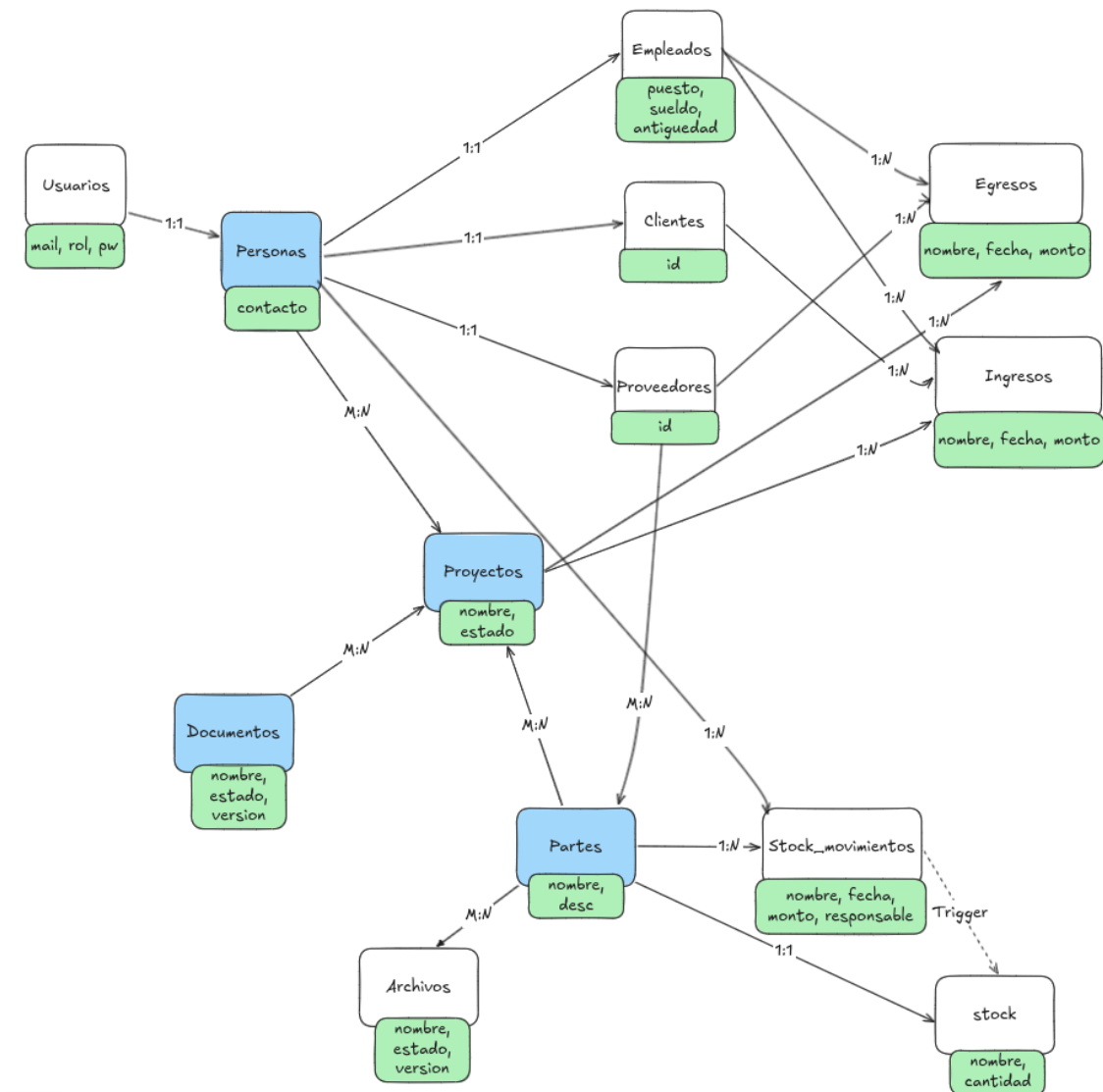
- Registro de egresos
- Registro de ingresos
- Stock

Datos de Auditoría

- Versiones de documentos
- Versiones de archivos
- Movimientos de Stock

Para la demostración se generarán datos falsos para completar las distintas tablas y validar el funcionamiento del sistema.

3. Modelo conceptual

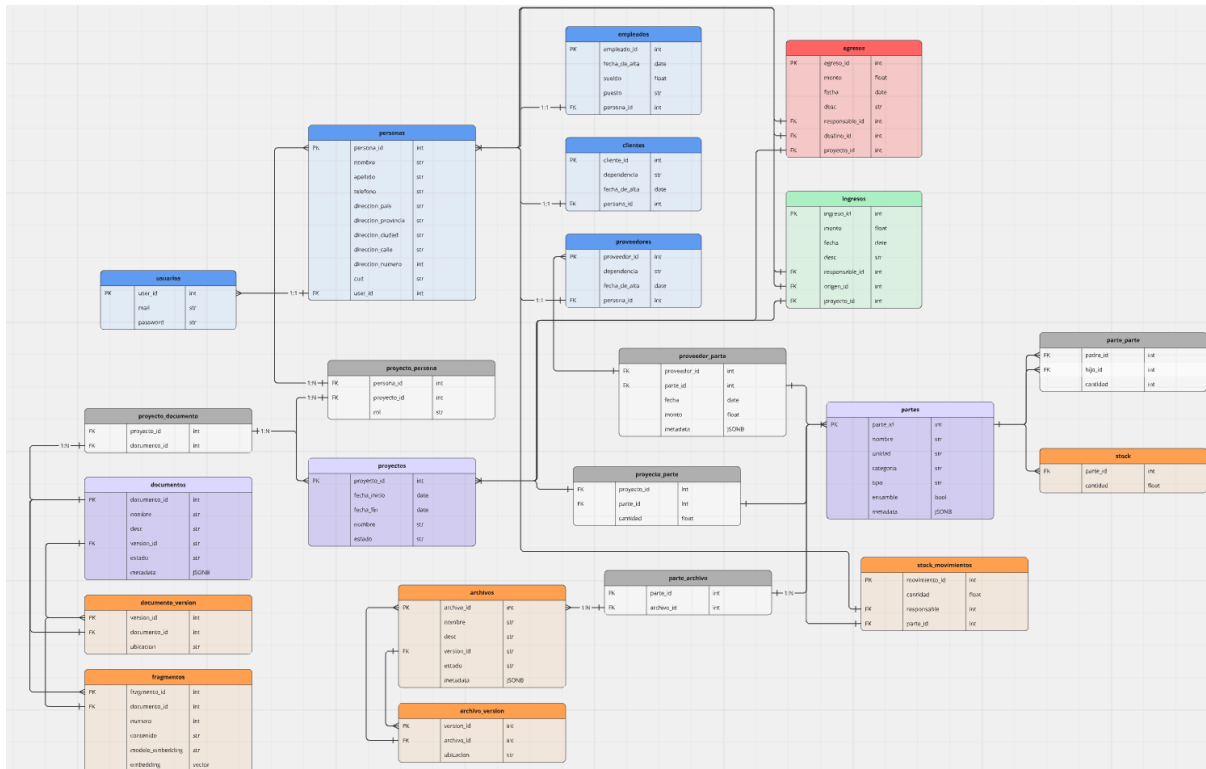


A la hora de hacer el modelo conceptual, se tomaron distintas decisiones pensando en la compatibilidad a futuro. Se enumeran los siguientes criterios adoptados.

- Se crea la entidad personas, que contiene información para identificar a una única persona real o jurídica. No se le asigna un rol directamente, sino que se lo corresponde a una de las tablas de clientes, proveedores o empleados. La motivación de esto es resolver la ambigüedad entre una persona que puede ser proveedor y cliente.
- Se crea la entidad usuarios que define las credenciales y niveles de acceso y se liga a una única persona real/jurídica. Esto permite la idea de tener una cuenta por persona y poder a futuro brindar acceso a personas no pertenecientes a la organización asignando un nivel de acceso.
- Se crean las entidades de ingresos y egresos en pos de una unificada con la motivación de poder establecer esquemas distintos para cada caso.

- Se crea la entidad proyectos, con el fin de poder asociar información a este. La idea es relacionar personas implicadas, gastos producidos, ingresos producidos, partes que utiliza y documentos relacionados.
- Se crea la entidad partes, pensada como un registro de partes que componen a un proyecto, pudiendo ser tanto comerciales como personalizadas. Una decisión fue la de dejar las partes como una entidad global, entendiéndose por oposición a una entidad que dependa del proyecto en particular. Esto último se infunde en que permitirá tener un control de stock más simple a la vez que se podrán replicar partes en distintos proyectos sin necesidad de crear nuevas. Estas partes pueden tener asociada uno o varios archivos como planos, especificaciones técnicas, etc. También se permite asociar las partes con los proveedores para tener registro de donde se pueden conseguir y una entidad partes_relacion que permite manejar composiciones de ensambles.
- Se crea la entidad de stock con el fin de poder monitorear las existencias de partes que hay actualmente. Se decidió que no sea modificada directamente, si no que sea stock_movimientos la entidad empleada para ello a los fines de tener trazabilidad.
- Se crea la entidad documentos, entendiéndose como aquellos archivos de texto escrito que pueden o no estar vinculados a uno o más proyectos. Estos pueden ser normativas, planificaciones, reclamos, etc. y la idea es poder llevar un registro de versiones de los mismos mediante la entidad así como también representarlos de forma vectorial para su uso en sistemas RAG.

4. Modelo lógico



5. Normalización, desnormalización y decisiones de diseño

En el sistema propuesto, se usa mucha normalización, con el fin de tener mejor control sobre datos duplicados así como también operaciones de borrado.

En particular podemos destacar los siguientes ejemplos:

- Separar users de personas.
- Separar personas de clientes, proveedores y empleados.
- Separar partes y stock
- Separar direccion de persona en varios campos
- Separar documentos de fragmentos
- Definir ensambles mediante parte_parte

Volviendo a la propuesta original, las decisiones de diseño permiten acceder a los datos de la forma esperada:

- Llevar registro de los stakeholders: Se logra a través de consultar las tablas de clientes, proveedores y empleados.
- Llevar registro de compras y ventas: Se logra a través de consultar las tablas de ingresos y egresos.

- Llevar registro de proyectos: Se logra consultado informacion propia del proyecto en la tabla proyectos, asi como sus relaciones mediante las tablas proyecto_documento, proyecto_persona, proyecto_parte, ingresos y egresos.
- Llevar control de stock: Se logra consultado directamente la tabla stock y su historial en la tabla stock_movimientos.
- Llevar control de versiones: Se logra mediante las tablas documento_version y archivo_version, permitiendo mantener distintas versiones en caso de que se necesite revisar cambios.
- Consultas de documentacion: Se logra mediante la tabla de fragmentos, que permite ser consultada de forma vectorial.

6. Selección tecnológica

En este caso de uso se propone utilizar postres acompañada de un object store como minio.

La motivación principal de usar postres radica en su versatilidad para representar múltiples tipos de datos en una sola base de datos. En la aplicación desarrollada se observan:

- Datos estructurados que son fácilmente representados mediante la base de datos relacional.
- Datos no estructurados en baja cantidad, previstos para dar flexibilidad a la hora de registrar documentos y partes que fueron solventados mediante JSONB permitiendo esta flexibilidad así como la asociación con la base de datos relacional.
- Datos vectoriales para almacenar los embeddings de los documentos, que en este caso la extensión pgvector provee soporte de los mismos, con la ventaja de además mantener las asociaciones relacionales.

Debido a que en primer instancia se pensó como una herramienta interna, el volumen de datos no debería crecer de forma tal que se justifique alguna alternativa columnar o distribuida.

Por otro lado, se necesita acompañarla con minio para poder almacenar los documentos y los archivos, asociando las URIs de los mismos en los atributos correspondientes de las tablas de postgres.

7. Implementación mínima

Para la implementación mínima se incluye:

- Servicios de minio, postgres, pgadmin, fastAPI y nginx
- Creación de todas las tablas presentes en el modelo lógico, incluyendo relaciones, tipos de datos, restricciones y RLS.
- Creación de roles para acceso desde aplicación y agente de IA
- Endpoints en fastAPI para interactuar con la base de datos desde el front end servido por nginx.

En esta implementación no se incluye:

- Implementación de agente de IA para consultas en lenguaje natural.

- Consultas por búsqueda vectorial.
- Fragmentación y embeddings de documentos.

8. Consultas representativas

Se propone responder las siguientes consultas:

- Cuales son los proyectos activos?
- Donde se encuentra la ultima version del archivo x?
- Que componentes (desglosados) se requieren para llevar a cabo el proyecto x?
- Cuanto dinero ingreso y salio en el dia/mes/año.
- Que componentes faltan para llevar a cabo el proyecto x?
- Donde puedo comprar el proyecto x?
- Que archivos asociados tiene la parte x?

Las queries necesarias para responderlas se encuentran en el repositorio.

9. Propuesta para datos semiestructurados, no estructurados o vectoriales.

Como se menciono anteriormente, este proyecto cuenta con distintos tipos de datos, para su manejo se propuso:

- Encontramos datos semiestructurados en entidades que se benefician de cierta flexibilidad para representarlas, se dejo un campo de JSONB para cubrirlas. La idea es poder tener esa representatividad, sin perder la relacionabilidad ni tener que incorporar otra base de datos no relacional aparte. Ejemplos de esto se observa en los atributos
 - metadata de la tabla archivos.
 - metadata de la tabla documentos.
 - metadata de la tabla partes.
 - metadata de la tabla proveedor_parte
- En cuanto a los datos no estructurados, se propuso incorporar un servicio de minio para almacenarlos, siguiendo un formato de carpetas que permite el versionado con el formato {bucket}/{id}/{version}. Luego las URIs correspondientes se almacenan en la base de datos relacional, lo que permite vincularlas para mantener el registro y facilitar el acceso. Ejemplos de estas son:
 - Archivos
 - Documentos
- Por ultimo en lo que respecta a los datos vectoriales, solo tenemos una entidad que los incorpora, que es la de fragmentos a traves del atributo embeddings. En este caso se incorporo la extension de pg vector y se creo la tabla de fragmentos que permite almacenar el vector junto a metadata y relacion con el archivo asociado.

10. Arquitectura de datos

Este proyecto en concreto no maneja un gran volumen de datos, lo cual en cierta medida simplifica la arquitectura necesaria. Debido a que se necesita almacenar datos no estructurados, con su control de versiones, se propone utilizar un data lake. El mismo esta compuesto por

- Bucket de archivos: Se almacena con la estructura `/ {archivo_id} / {n_version} /`
- Bucket de documentos: Se almacena con la estructura `/ {doc_id} / {n_version} /`
- Bucket de reportes: Se almacenan reportes semanales y mensuales de ingresos, egresos, stocks y movimientos de stock.

Los datos pueden ingresar al sistema a traves de la capa de aplicacion. Esta debe permitir

- Creacion, modificacion y eliminacion de proyectos.
- Creacion, modificación y eliminacion de archivos.
- Creacion, modificación y eliminacion de partes.
- Creacion, modificación y eliminacion de presupuesto.
- Asociaciones entre archivos y partes
- Asociaciones entre partes y proyectos
- Asociaciones entre documentos y proyectos
- Asociaciones entre proyectos y personas.
- Registro de ingresos
- Registro de egresos
- Registros de movimiento de stock

Los datos pueden ser consultados a traves de la capa de aplicacion.

11. Seguridad, permisos y aislamiento

La estrategia de seguridad del sistema esta pensada sobre distintas capas:

- En primer lugar, a nivel de red se restringe unicamente el acceso de servicios que estenn en la misma red local que la base de datos, en este caso FastAPI para implementar los endpoints de acceso y PgAdmin para interactuar de forma mas facil con la misma.
- Se restringe el acceso a nivel de aplicacion, forzando la interaccion del usuario a traves de los endpoints definidos en la API.
- Se definen 3 tipos de usuarios:
 - VIEW_ONLY: Solo pueden ver informacion si mantiene relacion con la misma
 - FULL_ACCESS: Permiso de visualizacion y modificación irrestricto
 - RESTRICTED: Permiso de visualiazacion y modificación de informacion si mantienen relacion con la misma
- Se protegen los documentos con 3 niveles de acceso:
 - PUBLICO: Cualquiera puede verlo
 - PRIVADO: Solo quienes esten vinculados o sean usuarios FULL_ACCESS
- Se definen 3 roles en postgres con diferentes niveles de acceso.
 - Admin: Irrestricto
 - App: CRUD con RLS

- AI_agent: Solo Select y RLS
- Se implementa RLS sobre:
 - Documentos
 - Archivos
 - Empleados
- Las consultas realizadas a traves de la API son autenticadas mediante tokens JWT para identificar el id del usuario que las realiza. Esto permite ademas poder registrar su id en la carga de ingresos, egresos y movimiento de stock

12. Escalabilidad y rendimiento

En este proyecto, el volumen de datos es bajo, sin embargo con el tiempo es probable que haya una gran cantidad de almacenamiento dedicado a documentos y archivos. Debido a que estos son almacenados en un data lake, el escalamiento horizontal del mismo es trivial por lo que no deberia ser un problema para la infraestructura en general.

Un posible cuello de botello en el sistema puede deberse a que las políticas de RLS requieren hacer joins. Por ejemplo, si un usairo VIEW_ONLY quiere acceder a un documento, se debera hacer una query a proyectos_documentos, para obtener los proyectos asociados y luego hacer otra query para ver las personas asociadas al proyecto y recien ahi verificar si el id del usuario esta en esa lista. Como solucion se propone indexar estos ids para poder agilizar este proceso.